Segurança em APIs REST e Microsserviços — Documento de Contexto (Tema 1)

Oct 9, 2026 · @eduty

1. Sobre o seminário e este documento

Este documento é a base de estudo do grupo para o Tema 1: dele saem os slides, as falas e os exemplos de código.

Item	Definição
Tema	1 — Segurança em APIs REST e Microsserviços
Grupo	Guilherme, Eduardo, Lucas Marcelino e Ruan (o único grupo com 4 integrantes)
Data da apresentação	Nov 5, 2026 (quinta-feira), junto com os Temas 2 e 3
Duração	18 a 22 minutos
Escopo oficial	Autenticação via tokens, Rate Limiting, CORS, sanitização de payload e aspectos de arquitetura segura para APIs
Entrega	Slides enviados pelo SUAP, de preferência antes da apresentação

A estrutura exigida pelo professor tem quatro partes. A tabela mostra onde cada uma está neste documento.

Parte exigida	O que o professor pede	Seções deste documento
1. Introdução e Conceituação	O que é, que problema de segurança resolve, onde entra no ciclo de vida do software	2 a 7
2. Funcionamento e Arquitetura	Diagramas de fluxo de dados, protocolos, modelos de ameaça	6, 8 a 12
3. Aplicação Prática e Código	Implementação correta e mitigação de vulnerabilidades	13

Parte exigida	O que o professor pede	Seções deste documento
4. Conclusão	Conclusões e recomendações para desenvolvedores	14 e 15

Atenção à sobreposição com outros grupos. O Tema 2 (OAuth2, OpenID Connect e JWT) e o Tema 3 (criptografia e gestão de segredos) se apresentam no mesmo dia. O Tema 8 (validação de entradas e injeções) vem em 19/nov. O nosso recorte é a visão da API como um todo: tokens, rate limiting, CORS, payload e arquitetura. Onde encostar nesses temas, basta explicar o papel da peça na API, sem detalhar o protocolo.

2. Resumo executivo

Segurança em APIs REST e microsserviços é o conjunto de controles que garante que cada requisição venha de quem diz ser, peça só o que pode, em volume aceitável e com dados válidos, mesmo dentro da rede interna.

- **O que é.** APIs REST expõem dados e lógica de negócio por HTTP. Microsserviços dividem o sistema em vários serviços que conversam por essas APIs. A segurança deixa de ser um muro na borda e passa a ser verificada em cada chamada.
- **Que problema resolve.** Acesso a dados de terceiros, tokens forjados ou roubados, abuso por automação, leitura indevida pelo navegador e payloads maliciosos. A referência é o OWASP API Security Top 10, edição 2023.
- **Por que importa agora.** Em 2025, 87% das organizações pesquisadas pela Akamai tiveram incidente ligado a API, e a média diária de ataques a APIs subiu 113% em um ano.
- **Onde se aplica.** Em todo o ciclo de vida: requisitos, projeto, código, testes, implantação, operação e desativação. No Brasil, Pix e Open Finance são exemplos de APIs com segurança exigida por regulador.

Os cinco itens do escopo oficial e o que cada um responde:

Item	Pergunta que responde	Risco OWASP ligado	Seção
Autenticação via tokens	Quem está chamando, e o que pode fazer?	API1, API2, API5	8
Rate limiting	Quanto pode chamar?	API4, API6	9
CORS	Quais sites podem ler a resposta no navegador?	API8	10

Item	Pergunta que responde	Risco OWASP ligado	Seção
Sanitização de payload	O que foi enviado é válido e permitido?	API3, API7, API10	11
Arquitetura segura	Onde cada controle fica, e em quem confiar?	API8, API9	12

Mensagem central para a apresentação: as falhas de API mais caras são simples, quase sempre de autorização, e nenhum controle isolado as resolve. A defesa é em camadas.

3. História: de RPC a microsserviços

A segurança de APIs virou disciplina própria só em 2019, quase vinte anos depois do REST: primeiro as APIs se espalharam, depois vieram os vazamentos, e só então os padrões de defesa.

3.1 As cinco fases

Fase 1 — Antes do REST (anos 1990). Sistemas distribuídos conversavam por chamadas remotas de procedimento (RPC), com tecnologias como CORBA e DCOM. Em 1998 surgiram o XML-RPC e o SOAP, que levaram essas chamadas para cima do HTTP usando XML. A segurança era de perímetro: firewall na borda e confiança total na rede interna.

Fase 2 — Nasce o REST e a API web (2000 a 2006). Roy Fielding, coautor da especificação do HTTP, descreveu o REST em sua tese em 2000. No mesmo ano, Salesforce e eBay abriram as primeiras APIs web comerciais. A Amazon passou a exigir que suas equipes internas se comunicassem por interfaces de serviço, prática que abriu caminho para a AWS. O JSON substituiu aos poucos o XML por ser mais leve.

Fase 3 — Mobile e APIs públicas (2007 a 2012). O smartphone mudou o cliente: o aplicativo não recebe páginas prontas, ele consome uma API. Twitter, Facebook e Google abriram APIs para terceiros. Surgiu o problema de dar acesso a um aplicativo sem entregar a senha do usuário, resolvido pelo OAuth (2007) e pelo OAuth 2.0 (2012).

Fase 4 — Nuvem e microsserviços (2011 a 2017). Empresas como Netflix e Amazon quebraram seus monólitos em centenas de serviços para escalar equipes e implantações. O termo "microservices" apareceu em um encontro de arquitetos em 2011 e foi consolidado pelo artigo de James Lewis e Martin Fowler em 2014. Docker (2013) e Kubernetes (2014) tornaram viável operar tantos serviços. A rede interna deixou de ser confiável por definição.

Fase 5 — Segurança de API como disciplina (2018 em diante). Vazamentos causados por falhas simples de autorização em APIs se repetiram em empresas grandes. A OWASP criou um Top 10 só para APIs em 2019 e o atualizou em 2023. O NIST publicou guias para microsserviços (2019), Zero Trust (2020) e proteção de APIs (2025). Desde 2024, agentes de IA consomem APIs em escala, e relatórios de 2026 já tratam a API como a principal superfície de ataque.

3.2 Linha do tempo

Ano	Marco	Por que importa para o tema
1991	CORBA 1.0	Objetos distribuídos; complexo e preso à rede interna
1998	XML-RPC e primeiras versões do SOAP	Chamadas remotas passam a trafegar sobre HTTP
2000	Tese de Roy Fielding define o REST	Base conceitual de toda API moderna
2000	APIs web da Salesforce e do eBay	Primeiras APIs comerciais expostas na internet
2002	Amazon lança seus primeiros web services	API vira produto e modelo de organização interna
2006	APIs do Twitter e do Facebook; AWS S3 e EC2	APIs públicas em massa; nuvem sob demanda
2006	JSON publicado como RFC 4627	Formato leve que se torna padrão de payload
2007	OAuth 1.0; lançamento do iPhone	Acesso delegado sem compartilhar senha; apps consomem APIs
2011	Termo "microservices" surge em workshop de arquitetos; criação do Swagger	Nome para o novo estilo; documentação de API legível por máquina
2012	OAuth 2.0 (RFC 6749) e Bearer Token (RFC 6750)	Padrão atual de autorização para APIs
2013	Docker	Empacotar e isolar cada serviço fica barato
2014	Artigo "Microservices" de Lewis e Fowler; Kubernetes; CORS vira recomendação W3C; OpenID Connect 1.0	Ano em que o ecossistema atual se forma

Ano	Marco	Por que importa para o tema
2015	JWT (RFC 7519); OpenAPI Initiative; GraphQL e gRPC abertos	Token autocontido; contrato de API padronizado
2017	Istio (service mesh); OpenAPI 3.0	mTLS e políticas entre serviços sem mudar o código
2018	LGPD sancionada (Lei 13.709); vazamentos via API no Facebook e no serviço postal dos EUA	Vazar dado pessoal por API passa a ter consequência legal no Brasil
2019	OWASP API Security Top 10 (1ª edição); NIST SP 800-204	Primeiro catálogo de riscos específico de APIs; guia para microsserviços
2020	RFC 8725 (boas práticas de JWT); NIST SP 800-207 (Zero Trust); LGPD em vigor; lançamento do Pix	Pix e Open Finance levam APIs com mTLS e OAuth ao sistema financeiro brasileiro
2022	Vazamento da Optus (Austrália) por API sem autenticação	Caso clássico de API esquecida exposta à internet
2023	OWASP API Security Top 10 (2ª edição); vazamento da T-Mobile via API (37 milhões de contas)	Edição vigente do Top 10
2025	RFC 9700 (boas práticas de segurança do OAuth 2.0); NIST SP 800-228 (proteção de APIs); OWASP Top 10:2025	Guias mais recentes
2026	Atualização do NIST SP 800-228 (março); relatório Akamai SOTI 2026	APIs descritas como principal superfície de ataque

4. Conceitos fundamentais

API REST é o contrato HTTP pelo qual sistemas trocam dados; microsserviço é a forma de dividir um sistema em vários serviços pequenos que conversam por essas APIs.

4.1 API

API (Application Programming Interface) é uma interface que permite a um software usar funções ou dados de outro sem conhecer seu código interno. Uma API web expõe essa interface pela rede, quase sempre sobre HTTP. O aplicativo do banco, o site de e-commerce e o painel administrativo são clientes diferentes da mesma API.

Para segurança, o ponto central é este: a API entrega **dados e lógica de negócio diretamente**, sem a camada de tela no meio. Tudo o que a interface gráfica esconde, a API mostra para quem souber montar a requisição.

4.2 REST

REST (Representational State Transfer) é um estilo arquitetural descrito por Roy Fielding em sua tese de doutorado, em 2000. Não é protocolo nem padrão: é um conjunto de restrições aplicadas sobre HTTP. Cada restrição tem uma consequência de segurança.

Restrição REST	O que significa	Consequência para segurança
Cliente-servidor	Cliente e servidor evoluem separados	O servidor nunca pode confiar em validação feita no cliente
Sem estado (stateless)	O servidor não guarda sessão; cada requisição traz tudo o que precisa	A credencial viaja em toda requisição, em geral como token
Cache	Respostas podem ser armazenadas por intermediários	Resposta com dado sensível precisa de `Cache-Control: no-store`
Interface uniforme	Recursos identificados por URL, manipulados por verbos HTTP	URLs previsíveis (`/pedidos/1001`) facilitam enumeração de IDs
Sistema em camadas	Proxies e gateways ficam entre cliente e servidor	Permite concentrar controles no gateway, mas cada camada precisa ser configurada
Código sob demanda (opcional)	O servidor pode enviar código ao cliente	Pouco usado em APIs; aumenta risco quando usado

4.3 Verbos e códigos HTTP que importam para segurança

Verbo	Uso	Seguro (não altera dados)	Idempotente
GET	Ler um recurso	Sim	Sim
POST	Criar um recurso ou disparar ação	Não	Não
PUT	Substituir um recurso inteiro	Não	Sim

Verbo	Uso	Seguro (não altera dados)	Idempotente
PATCH	Alterar parte de um recurso	Não	Não necessariamente
DELETE	Remover um recurso	Não	Sim
OPTIONS	Perguntar o que o servidor aceita (usado no preflight do CORS)	Sim	Sim

Código	Significado	Quando a API deve responder
400 Bad Request	Requisição malformada	Payload fora do schema
401 Unauthorized	Não autenticado	Token ausente, inválido ou expirado
403 Forbidden	Autenticado, mas sem permissão	Usuário tenta acessar recurso ou função de outro
404 Not Found	Recurso não existe	Também usado para não revelar que um recurso alheio existe
413 Content Too Large	Corpo grande demais	Payload acima do limite
415 Unsupported Media Type	Formato não aceito	`Content-Type` diferente do esperado
422 Unprocessable Content	Formato certo, conteúdo inválido	Regra de negócio violada
429 Too Many Requests	Limite de requisições excedido	Rate limiting acionado
500 Internal Server Error	Falha no servidor	Sempre com mensagem genérica, sem stack trace

A diferença entre 401 e 403 costuma cair em perguntas: 401 é "não sei quem você é"; 403 é "sei quem você é e a resposta é não".

4.4 Microsserviços

Microsserviços são um estilo de arquitetura em que a aplicação é um conjunto de serviços pequenos. Cada serviço roda em seu próprio processo, cuida de uma capacidade de negócio, tem seu próprio banco de dados e é implantado de forma independente. Os serviços se comunicam por mecanismos leves, em geral APIs HTTP ou mensageria.

O oposto é o monólito: uma única aplicação, implantada inteira, em que os módulos se chamam por funções dentro da mesma memória.

Aspecto	Monólito	Microsserviços
Comunicação interna	Chamada de função em memória	Chamada de rede (HTTP, gRPC, fila)
Perímetro	Um ponto de entrada, uma borda clara	Dezenas ou centenas de APIs, borda difusa
Identidade	Sessão única do usuário	Identidade do usuário e identidade de cada serviço
Dados	Um banco	Um banco por serviço
Segredos	Poucos, em um lugar	Muitos, espalhados por serviços
Logs	Um arquivo, uma linha do tempo	Logs distribuídos, exigem correlação
Correção de falha	Reimplantar tudo	Reimplantar só o serviço afetado
Impacto de invasão	Tudo ou nada	Depende do isolamento entre serviços

A frase que resume o problema: **no monólito, a chamada entre módulos não passa pela rede; nos microsserviços, toda chamada interna é uma requisição que pode ser interceptada, forjada ou repetida.**

4.5 Outros estilos de API

O tema é REST, mas vale saber situar os vizinhos caso o professor pergunte.

Estilo	Formato	Onde aparece	Ponto de atenção
SOAP	XML, contrato WSDL	Sistemas legados, governo, bancos	Ataques a XML (XXE), padrão WS-Security pesado
REST	JSON sobre HTTP	Padrão de APIs públicas e móveis	Autorização por objeto, exposição excessiva de dados
GraphQL	Consulta flexível em um único endpoint	Front-ends ricos	Consultas profundas que esgotam recursos, introspecção exposta

Estilo	Formato	Onde aparece	Ponto de atenção
gRPC	Binário (Protobuf) sobre HTTP/2	Comunicação entre microsserviços	Menos ferramentas de inspeção, exige mTLS bem configurado
Webhooks	Chamada HTTP no sentido inverso	Notificações de pagamento, integrações	Validar assinatura de quem chama

5. O problema de segurança

A API é hoje o principal alvo de ataques na web: em 2025, 87% das organizações pesquisadas pela Akamai relataram ao menos um incidente de segurança ligado a API.

5.1 Por que APIs são alvo

- **Entregam o dado puro.** A resposta JSON traz registros completos, sem a tela filtrando o que aparece.
- **O cliente não é confiável.** O atacante não usa o aplicativo: ele usa curl, Postman ou um proxy como o Burp e monta a requisição que quiser.
- **São previsíveis e documentadas.** URLs seguem padrão, e a documentação (Swagger/OpenAPI) descreve cada endpoint. O que ajuda o desenvolvedor ajuda o atacante.
- **São muitas.** Cada funcionalidade vira vários endpoints, em várias versões. Versões antigas e APIs esquecidas continuam no ar.
- **O ataque parece tráfego normal.** Pedir `/pedidos/1002` em vez de `/pedidos/1001` não tem assinatura maliciosa. Um firewall de aplicação que procura padrões de injeção não vê nada errado.
- **São fáceis de automatizar.** Um script percorre milhões de IDs se ninguém limitar a taxa.
- **Microsserviços multiplicam tudo isso.** Cada serviço expõe sua própria API interna, e a rede interna vira parte da superfície de ataque.

5.2 Que problema a segurança de APIs resolve

Cada controle do nosso escopo protege uma propriedade clássica da segurança da informação.

Propriedade	Ameaça típica em APIs	Controle do nosso escopo
Confidencialidade	Ler dados de outro usuário trocando um ID; resposta com campos demais	Autenticação por token, autorização por objeto, filtro de saída
Integridade	Alterar campos que o usuário não deveria (ex.: `isAdmin`), injeção	Validação e sanitização de payload
Disponibilidade	Enxurrada de requisições, consultas caras, payloads gigantes	Rate limiting, limites de tamanho e paginação
Autenticidade	Token forjado, serviço falso na rede interna	Validação de assinatura do token, mTLS entre serviços
Não repúdio e auditoria	Ação sem rastro	Logs estruturados com identificador de correlação

O CORS entra como proteção do **usuário no navegador**: impede que um site malicioso leia respostas da API usando a sessão da vítima.

5.3 Números recentes

Os dados abaixo são do relatório State of the Internet 2026 da Akamai, publicado em 17 de março de 2026. É um relatório de fornecedor de segurança: útil para dimensionar o problema, mas vale citar a fonte ao apresentar.

Indicador	Valor	Período
Organizações pesquisadas com incidente de segurança em API	87%	2025
Aumento da média diária de ataques a APIs	113%	2025 sobre 2024
Aumento de ataques DDoS na camada 7 (aplicação)	104%	últimos dois anos
Aumento de ataques a aplicações web	73%	2023 a 2025

Fonte: Akamai, comunicado do relatório SOTI 2026. O mesmo relatório aponta que código gerado por IA sem revisão ("vibe coding") vem levando falhas e configurações erradas à produção.

5.4 Casos reais

Nenhum destes casos exigiu técnica avançada. Todos são falhas dos itens do nosso escopo.

Caso	O que aconteceu	Falha	Lição
Optus (Austrália, set/2022)	Um erro de código de 2018 deixou sem efeito o controle de acesso de uma API. A empresa corrigiu o domínio principal em 2021, mas não o subdomínio `api`, que não era usado desde 2017. Dados de cerca de 9,5 milhões de clientes foram obtidos em três dias.	Autenticação quebrada e API esquecida no ar	Manter inventário de APIs; corrigir a falha em todos os pontos de entrada
T-Mobile (EUA, nov/2022 a jan/2023)	Um invasor extraiu dados de cerca de 37 milhões de contas por uma única API, durante aproximadamente seis semanas até ser detectado.	Causa técnica não divulgada pela empresa; a extração durou semanas sem ser detectada	Rate limiting e alerta de comportamento anômalo
Dell (mai/2024)	Segundo o autor do ataque, ele se cadastrou como parceiro com empresas falsas e percorreu códigos de equipamento em uma API do portal, a 5.000 requisições por minuto por três semanas. Foram 49 milhões de registros.	Sem rate limiting; sem checagem de qual parceiro pode ver qual cliente	Token válido não basta: é preciso autorização por objeto e limite de taxa

Caso	O que aconteceu	Falha	Lição
Capital One (EUA, 2019)	Uma falha de SSRF fez um servidor interno buscar credenciais temporárias da nuvem, usadas para copiar dados de mais de 100 milhões de pessoas.	Requisição forjada pelo servidor (SSRF)	Validar URLs recebidas; restringir o que o serviço pode acessar

O regulador australiano descreveu o ataque à Optus como algo que não exigia habilidade avançada nem conhecimento interno. Essa é a melhor frase para a introdução: **as falhas de API mais caras são as mais simples.**

5.5 Consequência legal no Brasil

Vazamento de dado pessoal por API é incidente de segurança nos termos da LGPD (Lei 13.709/2018).

- **Art. 46:** o controlador deve adotar medidas de segurança desde a concepção do produto até a sua execução.
- **Art. 48:** incidentes com risco relevante devem ser comunicados à ANPD e aos titulares.
- **Art. 52:** as sanções incluem multa de até 2% do faturamento, limitada a R$ 50 milhões por infração.

O Tema 6 aprofunda a LGPD. Para nós basta a ligação: uma API sem autorização por objeto é, juridicamente, ausência de medida de segurança.

6. Modelo de ameaças

O modelo de ameaças de referência para APIs é o OWASP API Security Top 10, edição 2023, que continua sendo a versão vigente em outubro de 2026.

6.1 OWASP API Security Top 10 (2023)

A OWASP é uma fundação sem fins lucrativos dedicada à segurança de aplicações. O Top 10 de APIs é separado do Top 10 tradicional de aplicações web porque os riscos mudam quando a lógica de negócio fica exposta por interfaces programáticas.

ID	Risco	Em uma frase	Exemplo	Item do nosso escopo
API1	Broken Object Level Authorization (BOLA)	A API não confere se o objeto pedido pertence a quem pede	Trocar `/pedidos/1001` por `/pedidos/1002` e ver o pedido de outro cliente	Autorização junto ao token
API2	Broken Authentication	Falhas no mecanismo que identifica o cliente	Token sem expiração, login sem limite de tentativas, assinatura do JWT não validada	Autenticação via tokens
API3	Broken Object Property Level Authorization	A API expõe ou aceita campos que não deveria	Resposta traz `senhaHash`; requisição envia `"papel": "admin"` e a API grava	Sanitização de payload
API4	Unrestricted Resource Consumption	Sem limite de requisições, tamanho ou custo	`?limite=1000000`; milhares de SMS de verificação disparados, pagos por requisição	Rate limiting
API5	Broken Function Level Authorization	Usuário comum consegue chamar função administrativa	`DELETE /admin/usuarios/7` funciona com token de cliente	Autorização junto ao token
API6	Unrestricted Access to Sensitive Business Flows	Fluxo de negócio legítimo usado em excesso por automação	Robô compra todos os ingressos em segundos	Rate limiting por regra de negócio
API7	Server Side Request Forgery (SSRF)	A API busca uma URL informada pelo cliente sem validar	`{"avatarUrl": "http://169.254.169.254/..."}` faz o servidor acessar a rede interna	Sanitização de payload
API8	Security Misconfiguration	Configuração insegura em qualquer camada	CORS liberado para qualquer origem, stack trace na resposta, TLS antigo	CORS e arquitetura
API9	Improper Inventory Management	Ninguém sabe quais APIs e versões estão no ar	`/v1` antiga sem as correções da `/v2`; endpoint de debug exposto	Arquitetura
API10	Unsafe Consumption of APIs	Confiar em dados de APIs de terceiros sem validar	Serviço grava direto no banco o que um parceiro devolveu	Payload e arquitetura

Três dos cinco primeiros itens (API1, API3 e API5) são falhas de **autorização**. Autenticar bem não basta: a pergunta "quem é você" está respondida, falta responder "você pode fazer isto com este objeto".

6.2 O que mudou de 2019 para 2023

Mudança	Detalhe
Fusão	"Excessive Data Exposure" e "Mass Assignment" (2019) viraram um só item, API3:2023, porque a causa raiz é a mesma
Renomeado	"Lack of Resources & Rate Limiting" virou "Unrestricted Resource Consumption"
Entraram	Fluxos de negócio sensíveis (API6), SSRF (API7) e consumo inseguro de APIs (API10)
Saíram da lista	"Injection" e "Insufficient Logging & Monitoring" como itens próprios

Injeção saiu da lista de APIs, mas não deixou de existir: continua no Top 10 geral da OWASP (A05 na edição 2025) e é o assunto do Tema 8.

6.3 STRIDE aplicado a uma API

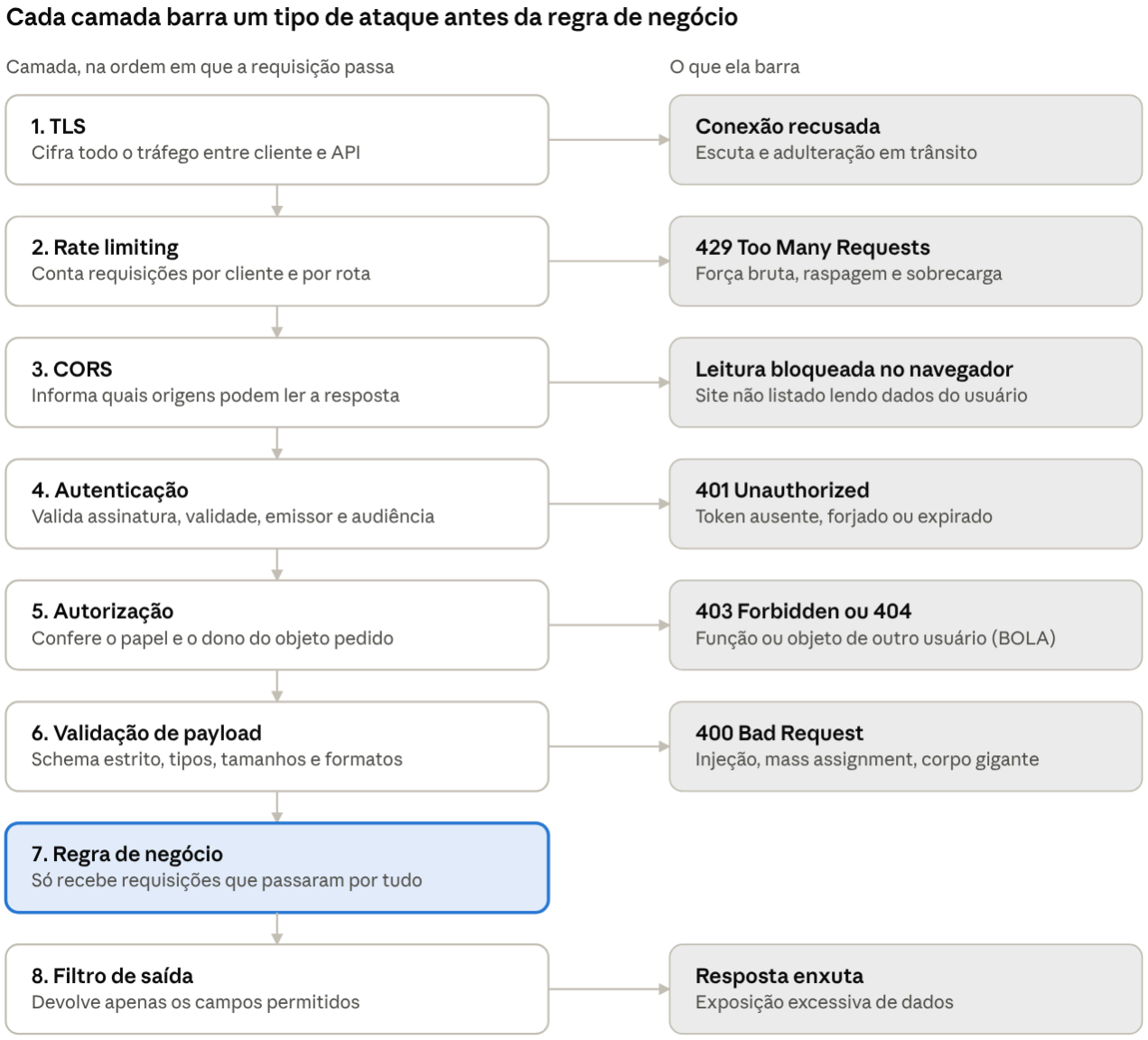
STRIDE é um método de modelagem de ameaças criado na Microsoft. Cada letra é uma categoria de ameaça. Serve para o grupo mostrar que sabe pensar em ameaças antes de escrever código.

Letra	Ameaça	Em uma API REST	Controle
S	Spoofing (falsificar identidade)	Token roubado ou forjado; serviço falso na rede interna	Validação de token, mTLS
T	Tampering (adulterar dados)	Payload alterado em trânsito ou com campos extras	TLS, validação por schema
R	Repudiation (negar a autoria)	Operação sem registro de quem fez	Logs com usuário, horário e ID de correlação
I	Information disclosure (vazar informação)	BOLA, resposta com campos demais, erro com stack trace	Autorização por objeto, filtro de saída, erros genéricos

Letra	Ameaça	Em uma API REST	Controle
D	Denial of service (negar serviço)	Enxurrada de requisições, consulta sem paginação	Rate limiting, limites de tamanho, timeouts
E	Elevation of privilege (elevar privilégio)	Usuário comum chama rota de administrador; mass assignment de `papel`	Autorização por função, lista de campos permitidos

6.4 Defesa em camadas

Nenhum controle sozinho cobre o Top 10. A requisição atravessa várias camadas, e cada uma barra um tipo de ataque.



camadas de defesa · 8 etapas e o que cada uma barra

A requisição desce pelas camadas e pode ser recusada em qualquer uma; só chega à regra de negócio o que passou por todas. As camadas 1 a 3 costumam ficar no gateway, a 4 acontece no gateway e de novo no serviço, e as camadas 5 a 8 ficam dentro do serviço.

7. Onde se aplica no ciclo de vida do software

Segurança de API se aplica em todas as fases do ciclo de vida, do requisito à desativação; quanto mais cedo a falha é encontrada, mais barato é corrigi-la (princípio do "shift left").

7.1 Fase a fase

Fase	O que fazer para a API	Ferramentas e artefatos
Requisitos	Classificar os dados que a API expõe; definir quem pode acessar o quê; escrever casos de abuso ("e se o usuário trocar o ID?")	Matriz de perfis e permissões, requisitos LGPD
Projeto e arquitetura	Modelar ameaças (STRIDE); escrever o contrato antes do código; escolher fluxo de autenticação; definir política de rate limit e de CORS	Especificação OpenAPI com esquemas de segurança, diagrama de fluxo de dados
Implementação	Validar toda entrada por schema; checar autorização em cada rota; nunca deixar segredo no código	Bibliotecas de validação, middleware de autenticação, revisão de código
Testes	Testar com dois usuários (A tenta acessar dado de B); análise estática e dinâmica; fuzzing de payload	SAST, DAST (OWASP ZAP), testes de contrato, OWASP crAPI para treino
Implantação	Configurar gateway, TLS, cabeçalhos e segredos; varrer imagens e dependências	Pipeline CI/CD, cofre de segredos, API gateway
Operação	Monitorar volume e erros 401/403/429; ajustar limites; responder a incidentes	Logs centralizados, alertas, WAF/WAAP

Fase	O que fazer para a API	Ferramentas e artefatos
Manutenção e desativação	Manter inventário de APIs e versões; desligar versões antigas; rotacionar chaves	Catálogo de APIs, política de depreciação

O guia NIST SP 800-228 organiza os controles de API da mesma forma, em dois momentos: antes da execução (projeto, código, testes) e em tempo de execução (gateway, monitoramento). O caso Optus é o exemplo de falha na última fase: uma API sem uso desde 2017 continuou exposta até 2022.

As fases de testes e implantação se cruzam com o Tema 5 (SAST e DAST no pipeline DevSecOps).

7.2 Onde se aplica na prática

Contexto	Exemplo	Controle que mais pesa
API pública para aplicativo móvel ou SPA	App de delivery, internet banking	Tokens de curta duração, autorização por objeto, CORS
API para parceiros (B2B)	Marketplace integrando lojistas	Chaves por parceiro, escopos, rate limit por cliente
Comunicação interna entre microsserviços	Serviço de pedidos chama serviço de pagamento	mTLS, identidade de serviço, Zero Trust
Sistema financeiro brasileiro	Pix e Open Finance	OAuth 2.0 com mTLS, exigidos pelo Banco Central
Internet das Coisas	Sensores enviando leituras	Credencial por dispositivo, limites de taxa
Agentes de IA consumindo APIs	Assistente que executa ações em sistemas	Escopos mínimos, limites por agente, auditoria

O Pix é um bom exemplo local para o slide. O manual da API Pix do Banco Central exige conexão TLS 1.2 ou superior, OAuth 2.0 no fluxo Client Credentials com TLS mútuo (mTLS, RFC 8705) e escopos que separam as funcionalidades por perfil. A documentação de quem implementa a API reproduz esses requisitos, como a da Cielo.

7.3 Como aplicar: ordem sugerida

1. Inventariar as APIs existentes e os dados que cada uma expõe.
2. Colocar todas atrás de HTTPS e de um ponto único de entrada (gateway).

3. Exigir autenticação em todo endpoint que não seja público por decisão explícita.
4. Implementar autorização por objeto e por função em cada rota.
5. Validar entradas por schema e filtrar campos de saída.
6. Definir rate limiting por usuário e por rota sensível.
7. Configurar CORS com lista de origens permitidas.
8. Centralizar logs e criar alertas.
9. Automatizar testes de segurança no pipeline.
10. Rever o inventário a cada versão e desligar o que não é usado.

8. Autenticação via tokens e autorização

Como a API REST não guarda sessão, cada requisição precisa provar quem a envia; o token é essa prova, e a autorização decide o que o portador pode fazer.

8.1 Autenticação x autorização

	Autenticação	Autorização
Pergunta	Quem é você?	Você pode fazer isto?
Quando ocorre	No início da requisição	Depois da autenticação, em cada operação
Falha responde	401 Unauthorized	403 Forbidden
Risco OWASP	API2	API1, API3, API5
Onde costuma ficar	Gateway ou middleware	Dentro do serviço, junto à regra de negócio

8.2 Tipos de credencial

Credencial	Como funciona	Vantagens	Desvantagens	Uso típico
Basic Auth	Usuário e senha em Base64 a cada requisição	Simples	Senha trafega sempre; sem expiração	Evitar; só em legado sobre TLS

Credencial	Como funciona	Vantagens	Desvantagens	Uso típico
API key	Chave fixa no cabeçalho	Simples; identifica o sistema cliente	Não identifica usuário; longa duração; vaza em repositórios	Integração entre sistemas, junto a outro controle
Cookie de sessão	ID de sessão guardado no servidor	Revogação imediata	Exige estado no servidor; sujeito a CSRF	Aplicação web tradicional
Token opaco	Texto aleatório; a API consulta o emissor para validar	Revogação imediata; nada legível no token	Uma consulta extra por requisição	APIs que precisam de revogação forte
JWT	Token assinado que carrega os dados do usuário	Validação local, sem consulta; bom para microsserviços	Difícil de revogar antes de expirar; conteúdo legível	APIs e microsserviços em geral
Certificado (mTLS)	Cliente apresenta certificado na conexão TLS	Identidade forte, difícil de roubar	Gestão de certificados trabalhosa	Serviço a serviço; setor financeiro

8.3 Anatomia de um JWT

JWT (JSON Web Token, RFC 7519) tem três partes separadas por ponto: cabeçalho, conteúdo e assinatura. As duas primeiras são JSON codificado em Base64URL.

```
// Cabeçalho
{ "alg": "RS256", "typ": "JWT", "kid": "chave-2026-10" }

// Conteúdo (claims)
{
  "iss": "https://auth.exemplo.com",
  "sub": "usuario-4821",
  "aud": "api-pedidos",
  "exp": 1791500400,
  "iat": 1791499500,
  "jti": "b7f3c2e1",
  "scope": "pedidos:ler pedidos:criar"
}
```

Claim	Significado	A API deve
`iss`	Quem emitiu	Conferir se é o emissor esperado
`sub`	De quem é o token	Usar como identidade do usuário
`aud`	Para qual API foi emitido	Rejeitar se não for ela
`exp`	Quando expira	Rejeitar se já passou
`iat`	Quando foi emitido	Usar em auditoria
`jti`	Identificador único	Usar em lista de revogação
`scope`	Permissões concedidas	Conferir antes de cada operação

O JWT é assinado, não cifrado. Qualquer pessoa com o token lê o conteúdo. Nunca colocar senha, CPF ou dado sensível dentro dele.

Algoritmo	Tipo	Quem valida precisa de	Indicado para
HS256	Simétrico (segredo compartilhado)	O mesmo segredo que assina	Um único serviço emite e valida
RS256 / ES256	Assimétrico (chave privada e pública)	Só a chave pública	Microsserviços: um emite, vários validam

8.4 Fluxo de uma requisição autenticada

1. O usuário se autentica no servidor de autorização (login, com segundo fator quando possível).
2. O servidor devolve um **access token** de vida curta (minutos) e um **refresh token** de vida mais longa.
3. O cliente envia o access token em cada chamada: `Authorization: Bearer <token>`.
4. A API valida assinatura, `exp`, `iss` e `aud`.
5. A API confere a autorização para a operação e o objeto pedidos.
6. Quando o access token expira, o cliente troca o refresh token por um novo par. O refresh token antigo é invalidado (rotação).

8.5 Erros comuns com tokens

Erro	Consequência	Correção
Aceitar `"alg": "none"` ou o algoritmo que o token declara	Atacante forja tokens sem assinatura válida	Fixar no código a lista de algoritmos aceitos
Não validar `exp`	Token roubado vale para sempre	Expiração curta e sempre verificada
Não validar `aud` e `iss`	Token emitido para outro sistema é aceito	Conferir os dois em toda requisição
Segredo HS256 fraco ou no repositório	Qualquer um assina tokens	Segredo longo e aleatório, guardado em cofre
Token na URL (`?token=...`)	Fica em logs, histórico e cabeçalho Referer	Sempre no cabeçalho `Authorization`
Guardar token em `localStorage`	Um XSS rouba o token	Cookie `HttpOnly`, `Secure`, `SameSite`, ou token só em memória
Sem estratégia de revogação	Logout não invalida nada	Vida curta, rotação de refresh token, lista de `jti` revogados
Dado sensível no conteúdo	Vazamento por simples leitura	Só identificadores e permissões

A RFC 8725 reúne as boas práticas de JWT. As correções da tabela seguem o que ela recomenda.

8.6 OAuth 2.0 em um parágrafo

OAuth 2.0 (RFC 6749) é o padrão para delegar acesso: um servidor de autorização emite tokens com escopos limitados, e a API (servidor de recursos) apenas os valida. Para usuários, o fluxo recomendado é Authorization Code com PKCE. Para comunicação entre sistemas, Client Credentials. A RFC 9700, de janeiro de 2025, reúne as boas práticas atuais e desaconselha os fluxos implícito e de senha. O OAuth 2.1 consolida tudo isso, mas ainda é rascunho na IETF em 2026. O detalhe dos fluxos é do Tema 2.

8.7 Autorização: o item mais explorado

- **Por objeto (BOLA).** Em toda rota que recebe um ID, conferir se o objeto pertence ao usuário do token. A consulta ao banco deve incluir o dono: buscar por `id` **e** `usuarioId`.
- **Por função (BFLA).** Rotas administrativas exigem papel ou escopo específico. A regra é negar por padrão.
- **Por propriedade.** Nem todo campo é de todos: o cliente vê o pedido, mas não a margem de lucro.
- **RBAC e ABAC.** RBAC decide pelo papel (cliente, gerente, admin). ABAC decide por atributos (dono do recurso, horário, filial). APIs reais combinam os dois.
- **IDs imprevisíveis ajudam, mas não resolvem.** UUID no lugar de número sequencial dificulta a enumeração. A checagem de dono continua obrigatória.

8.8 Tokens em microsserviços

Decisão	Opções	Recomendação
Onde validar o token	Só no gateway, ou no gateway e em cada serviço	Nos dois: o serviço não deve confiar que a chamada veio do gateway
Como o serviço B sabe quem é o usuário	Repassar o token original, ou trocar por um token interno	Token interno de vida curta, com audiência restrita ao serviço B
Como o serviço B sabe que quem chama é o serviço A	Rede confiável, API key ou mTLS	mTLS: cada serviço tem certificado próprio
Onde decidir a autorização	Gateway (grossa) e serviço (fina)	Gateway confere escopo; serviço confere dono do objeto

9. Rate limiting

Rate limiting é limitar quantas requisições um cliente pode fazer em um intervalo de tempo; protege a disponibilidade, o custo e os fluxos que não podem ser automatizados.

9.1 O que ele resolve

Ataque ou abuso	Como o limite ajuda
Negação de serviço na camada de aplicação	Corta o excesso antes de chegar ao banco de dados
Força bruta em login e códigos de verificação	Poucas tentativas por minuto tornam o ataque inviável
Enumeração de IDs e raspagem de dados (scraping)	Percorrer milhões de registros passa a levar anos
Abuso de fluxo de negócio	Limita compras, cadastros ou cupons por conta
Custo descontrolado	Evita milhares de SMS, e-mails ou chamadas pagas a terceiros
Cliente com defeito	Um laço infinito em um aplicativo não derruba a API para todos

O caso Dell mostra a conta: a 5.000 requisições por minuto, 49 milhões de registros saíram em três semanas. A 60 por minuto, o mesmo volume levaria mais de um ano e meio.

Termos próximos: **rate limiting** rejeita o excesso; **throttling** atrasa ou enfileira o excesso; **quota** é o total permitido em um período longo (por dia ou por mês), em geral ligado ao plano contratado.

9.2 Algoritmos

Algoritmo	Como funciona	Vantagens	Desvantagens
Janela fixa	Conta requisições por intervalo fechado (ex.: cada minuto do relógio)	Simples, pouca memória	Permite o dobro do limite na virada da janela

Algoritmo	Como funciona	Vantagens	Desvantagens
Janela deslizante com registro	Guarda o horário de cada requisição e conta as do último intervalo	Exato	Consome memória proporcional ao tráfego
Janela deslizante com contador	Média ponderada entre a janela atual e a anterior	Bom equilíbrio entre precisão e custo	Aproximado
Token bucket (balde de fichas)	Fichas são repostas em taxa fixa; cada requisição gasta uma; o balde tem capacidade máxima	Permite rajadas curtas, comportamento natural para usuários	Dois parâmetros para ajustar
Leaky bucket (balde furado)	Requisições entram em fila e saem em taxa constante	Suaviza o tráfego para o backend	Rajadas legítimas esperam ou são descartadas

Token bucket é o mais usado em gateways e provedores de nuvem.

9.3 Limitar por quê e onde

Chave do limite	Quando usar	Cuidado
Endereço IP	Rotas sem autenticação (login, cadastro)	Vários usuários compartilham um IP; botnets usam milhares
Usuário ou token	Rotas autenticadas	Só vale depois de validar o token
Chave de API ou cliente	Integrações com parceiros	Permite limites diferentes por plano
Rota	Operações caras ou sensíveis	Limite mais baixo que o geral
Combinação	Login: por IP **e** por conta alvo	Evita tanto força bruta quanto bloqueio de inocentes

Onde aplicar	O que barra	Observação
Borda (CDN, WAF)	Volume bruto, DDoS	Não conhece o usuário
API gateway	Limites por cliente, rota e plano	Ponto principal

Onde aplicar	O que barra	Observação
Aplicação	Regras de negócio (3 pedidos de senha por hora)	Conhece o contexto
Entre serviços	Um serviço não derruba o outro	Junto a timeouts e circuit breaker

Com várias instâncias da API, o contador precisa ficar em um armazenamento compartilhado, como o Redis. Contador em memória local multiplica o limite pelo número de instâncias.

Cuidado com o cabeçalho `X-Forwarded-For`: se a aplicação confiar nele sem estar atrás de um proxy conhecido, o atacante escolhe o IP que quiser e nunca atinge o limite.

9.4 Como responder

A resposta padrão é o código 429 com o cabeçalho `Retry-After`. Um rascunho da IETF define os cabeçalhos `RateLimit-Policy` e `RateLimit` para o servidor informar a política e o saldo. O rascunho ainda não virou RFC (versão 11, maio de 2026), e muitas APIs usam os antigos `X-RateLimit-*`.

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 30
RateLimit-Policy: "padrao";q=100;w=60
RateLimit: "padrao";r=0;t=30

{ "erro": "limite_excedido", "mensagem": "Tente novamente em 30 segundos." }
```

No exemplo, `q=100;w=60` significa 100 requisições a cada 60 segundos; `r=0;t=30` significa saldo zero e renovação em 30 segundos.

9.5 Limites além da taxa

O risco API4 fala de consumo de recursos, não só de requisições por minuto.

- Tamanho máximo do corpo da requisição e de arquivos enviados.
- Paginação obrigatória, com teto para o tamanho da página.
- Tempo máximo de execução por requisição e por consulta ao banco.
- Número máximo de itens em listas e de níveis de aninhamento no JSON.
- Teto de gastos e alerta de faturamento em serviços pagos por chamada.

9.6 Rotas que pedem limite próprio

Login, cadastro, recuperação de senha, envio e verificação de código, busca, exportação de dados e qualquer operação que custe dinheiro. Para login, combinar limite por IP, limite por conta e atraso crescente entre tentativas. O bloqueio definitivo da conta é arriscado: vira arma para travar a conta de outra pessoa.

9.7 Vantagens e desvantagens

Vantagens	Desvantagens e riscos
Protege a disponibilidade para todos os clientes	Limite mal calibrado bloqueia usuários legítimos
Torna força bruta e raspagem inviáveis	Por IP, pune quem compartilha rede (empresas, universidades)
Controla custos de infraestrutura e de terceiros	Atacante distribuído contorna limites por IP
Dá previsibilidade de carga ao backend	Exige armazenamento compartilhado e monitoramento
Viabiliza planos comerciais com quotas	Não substitui autorização: quem pode ver tudo vê devagar

10. CORS

CORS é o mecanismo pelo qual a API diz ao navegador quais outros sites podem ler suas respostas; ele protege o usuário no navegador, não a API contra acesso direto.

10.1 Antes do CORS: a política de mesma origem

Origem é a combinação de **esquema, host e porta**. Por padrão, o navegador aplica a Same-Origin Policy: um script carregado de uma origem não pode ler respostas de outra.

URL comparada com `https://loja.exemplo.com`	Mesma origem?	Motivo
`https://loja.exemplo.com/carrinho`	Sim	Só o caminho muda
`http://loja.exemplo.com`	Não	Esquema diferente
`https://api.exemplo.com`	Não	Host diferente
`https://loja.exemplo.com:8443`	Não	Porta diferente

Sem essa política, qualquer site aberto em outra aba poderia chamar a API do seu banco com os seus cookies e ler o extrato.

10.2 O que o CORS faz

CORS (Cross-Origin Resource Sharing) é uma **exceção controlada** à política de mesma origem. O front-end em `https://loja.exemplo.com` precisa chamar a API em `https://api.exemplo.com`. A API responde com cabeçalhos dizendo que aquela origem é permitida, e o navegador libera a leitura. O mecanismo virou recomendação do W3C em 2014 e hoje é mantido no padrão Fetch do WHATWG.

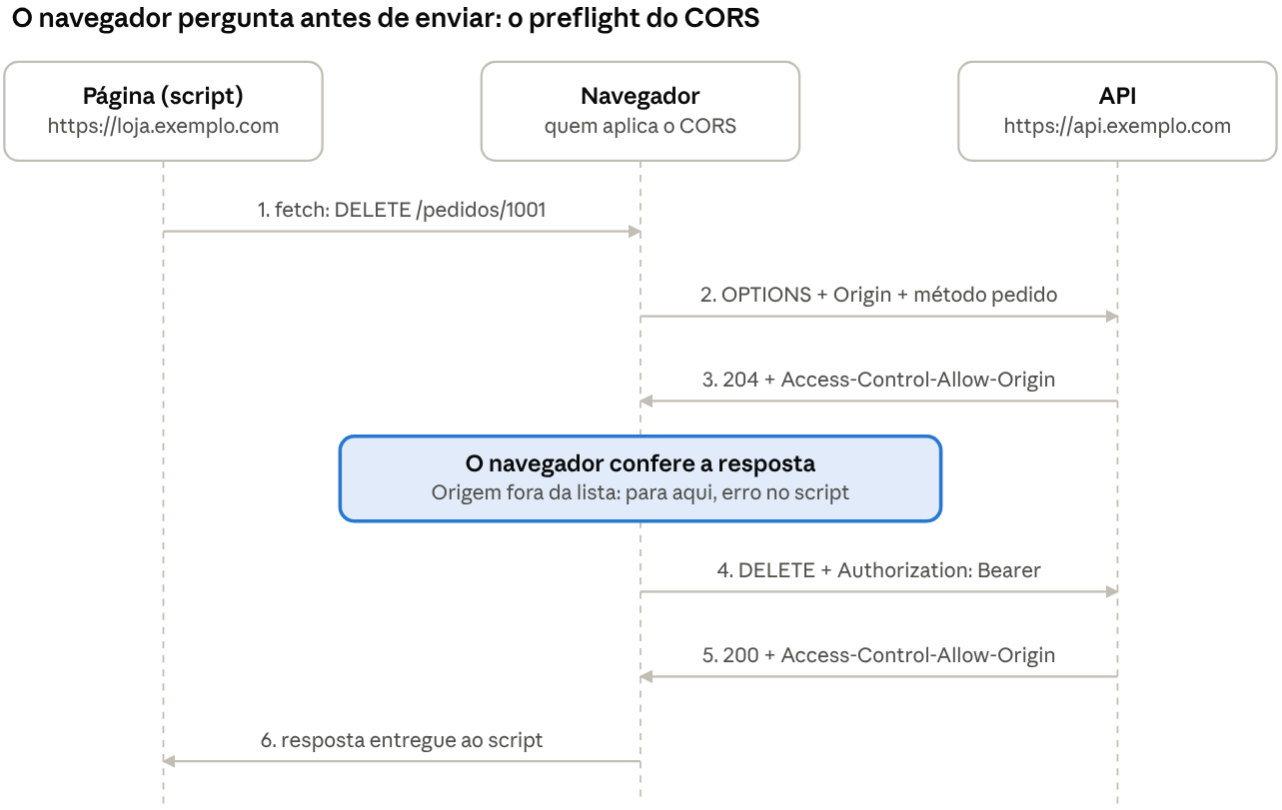
Três ideias que evitam erro na apresentação:

- **Quem aplica o CORS é o navegador.** curl, Postman, aplicativos móveis e outros servidores ignoram CORS. Ele não substitui autenticação.
- **CORS afrouxa, não aperta.** Sem nenhum cabeçalho CORS, o navegador bloqueia tudo entre origens. Cada cabeçalho adicionado abre uma exceção.
- **O erro de CORS no console é o navegador funcionando.** A solução nunca é "liberar tudo".

10.3 Requisição simples e preflight

Requisições que um formulário HTML antigo já conseguia fazer são chamadas simples: `GET`, `HEAD` ou `POST` com tipos de conteúdo de formulário. O navegador as envia direto e só decide depois se o script pode ler a resposta.

As demais passam antes por uma verificação chamada **preflight**. É o caso de `PUT`, `PATCH`, `DELETE`, de `Content-Type: application/json` e de qualquer requisição com o cabeçalho `Authorization`. Na prática, quase toda chamada a uma API REST tem preflight.



preflight do CORS · 6 mensagens, 1 decisão do navegador

A requisição real (passo 4) só sai do navegador se a resposta do preflight autorizar a origem, o verbo e os cabeçalhos. A API nunca vê o `DELETE` de um site não autorizado. As mensagens 2 e 3, em texto:

```
OPTIONS /pedidos/1001 HTTP/1.1
Host: api.exemplo.com
Origin: https://loja.exemplo.com
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://loja.exemplo.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Max-Age: 600
Vary: Origin
```

10.4 Cabeçalhos

Cabeçalho	Sentido	Função
`Origin`	Requisição	Origem da página que faz a chamada; preenchido pelo navegador
`Access-Control-Request-Method`	Preflight	Verbo que a requisição real vai usar
`Access-Control-Request-Headers`	Preflight	Cabeçalhos que a requisição real vai enviar
`Access-Control-Allow-Origin`	Resposta	Origem autorizada: uma origem exata ou `*`
`Access-Control-Allow-Methods`	Resposta	Verbos permitidos
`Access-Control-Allow-Headers`	Resposta	Cabeçalhos permitidos
`Access-Control-Allow-Credentials`	Resposta	`true` permite enviar cookies e ler a resposta
`Access-Control-Max-Age`	Resposta	Por quantos segundos o navegador guarda o resultado do preflight
`Access-Control-Expose-Headers`	Resposta	Cabeçalhos da resposta que o script pode ler
`Vary: Origin`	Resposta	Avisa os caches de que a resposta muda conforme a origem

10.5 Erros comuns

Erro	Por que é perigoso	Correção
Devolver no `Allow-Origin` qualquer `Origin` recebido, com `Allow-Credentials: true`	Qualquer site lê dados da API com a sessão da vítima	Comparar com uma lista fixa de origens
`Allow-Origin: *` em API com dados por usuário	Qualquer site lê respostas; o navegador só impede se houver credenciais	`*` apenas em API totalmente pública

Erro	Por que é perigoso	Correção
Validar a origem com "termina com" ou regex frouxa	`https://exemplo.com.atacante.net` ou `https://atacanteexemplo.com` passam	Comparação exata de texto
Aceitar a origem `null`	Páginas em sandbox e arquivos locais enviam `null`	Nunca incluir `null` na lista
Esquecer `Vary: Origin`	Cache entrega a um site a resposta liberada para outro	Sempre enviar quando a origem é dinâmica
Liberar todos os métodos e cabeçalhos	Superfície maior que o necessário	Listar só o que a API usa
Achar que CORS impede CSRF	Requisições simples são enviadas e executadas; só a leitura é bloqueada	Cookies com `SameSite` e token anti-CSRF

O navegador recusa a combinação `Allow-Origin: *` com `Allow-Credentials: true`. Para contornar o aviso, é comum o desenvolvedor passar a refletir a origem recebida, que é exatamente o primeiro erro da tabela.

10.6 CORS e CSRF

Se a API autentica por **token no cabeçalho `Authorization`**, um site malicioso não consegue anexar esse token: o navegador não o envia sozinho. Se a API autentica por **cookie**, o navegador o envia automaticamente, e um `POST` simples vindo de outro site é executado mesmo com CORS restrito. Nesse caso é preciso `SameSite` no cookie e token anti-CSRF.

10.7 Em microsserviços

CORS é assunto da borda. Só o gateway ou o BFF recebe chamadas de navegador, então a política fica em um lugar só. Serviços internos conversam entre si sem navegador e não precisam de CORS. Configurar no gateway e também no serviço gera cabeçalho duplicado, e o navegador rejeita a resposta.

10.8 Vantagens e desvantagens

Vantagens	Desvantagens e riscos
Permite front-end e API em domínios separados com segurança	Não protege a API de clientes fora do navegador

Vantagens	Desvantagens e riscos
Controle fino por origem, verbo e cabeçalho	Configuração confusa leva a "liberar tudo"
Padrão implementado por todos os navegadores	O preflight adiciona uma requisição extra
Mantém a política de mesma origem como padrão seguro	Erros de configuração são silenciosos: tudo funciona, inclusive para o atacante

11. Sanitização e validação de payload

Todo dado que chega à API é hostil até prova em contrário: a API valida o que entra contra um schema, rejeita o que sobra e filtra o que sai.

11.1 Três conceitos que se confundem

Conceito	O que faz	Exemplo
Validação	Confere se o dado obedece às regras e **rejeita** se não obedecer	`idade` deve ser inteiro entre 0 e 130; senão, 400
Sanitização	**Transforma** o dado para torná-lo seguro ou padronizado	Remover espaços, normalizar e-mail para minúsculas, remover tags HTML
Codificação de saída	Prepara o dado para o destino onde será usado	Escapar `<` como `<` ao montar HTML

A ordem de preferência é validar primeiro e sanitizar só o necessário. Sanitizar sem validar tenta consertar entrada ruim, e o atacante só precisa achar um caso que o filtro não previu.

Estratégia	Como decide	Avaliação
Lista de permissão (allow-list)	Aceita só o que está previsto	Preferida: o desconhecido é recusado
Lista de bloqueio (deny-list)	Recusa o que é sabidamente ruim	Frágil: sempre existe uma variação não listada

11.2 O que validar em cada requisição

Item	Regra	Risco evitado
`Content-Type`	Aceitar só o esperado (ex.: `application/json`)	Parsers alternativos, XXE
Tamanho do corpo	Limite em bytes	Esgotamento de memória
Estrutura	Schema com campos obrigatórios e opcionais	Dados incompletos ou malformados
Campos desconhecidos	Rejeitar	Mass assignment
Tipos	Texto, número, booleano, lista	Injeção NoSQL por objeto no lugar de texto
Formato	E-mail, UUID, data, CPF	Lixo no banco, injeção
Faixas e tamanhos	Mínimo e máximo para números, textos e listas	Abuso de recursos, estouro de lógica
Enumerações	Valor dentro de um conjunto fixo	Estados inválidos
Parâmetros de rota e de consulta	Mesmas regras do corpo	Enumeração, injeção
Arquivos	Tipo real, tamanho, nome	Upload malicioso, path traversal

A validação do front-end é experiência de uso. A do servidor é segurança. O atacante não usa o front-end.

11.3 Mass assignment

A API recebe um JSON e o grava inteiro no banco. O atacante acrescenta um campo que a tela não mostra.

```
PATCH /usuarios/eu HTTP/1.1
Content-Type: application/json

{ "nome": "Ana", "papel": "admin", "saldo": 99999 }
```

Se o código faz algo como `usuario.update(req.body)`, os três campos são gravados. A correção é copiar apenas os campos permitidos para aquela operação, ou rejeitar a requisição quando houver campo fora do schema.

11.4 Exposição excessiva de dados

É o mesmo problema na saída: a API devolve o registro inteiro do banco e confia que o front-end vai mostrar só o necessário.

```
{
  "id": 4821,
  "nome": "Ana",
  "email": "ana@exemplo.com",
  "senhaHash": "$2b$12$...",
  "cpf": "123.456.789-00",
  "papel": "cliente"
}
```

A correção é ter um formato de resposta explícito para cada rota e nunca serializar a entidade do banco diretamente. Mass assignment e exposição excessiva formam juntos o risco API3:2023.

11.5 Injeções pelo payload

O Tema 8 aprofunda este assunto. Para o nosso basta mostrar que o payload da API é o vetor.

Ataque	Payload de exemplo	Defesa
SQL injection	`"nome": "x' OR '1'='1"`	Consultas parametrizadas ou ORM; nunca concatenar texto
NoSQL injection	`"senha": { "$ne": null }`	Validar tipo: senha é texto, não objeto
Injeção de comando	`"arquivo": "a.png; rm -rf /"`	Não montar comandos de shell com entrada
XSS armazenado	`"comentario": "<script>...</script>"`	Codificar na saída; sanitizar HTML se for permitido
XXE	XML com entidade externa	Desativar entidades externas; preferir JSON
Poluição de protótipo (JavaScript)	`{ "__proto__": { "admin": true } }`	Rejeitar chaves desconhecidas; não mesclar objetos sem critério
Path traversal	`"arquivo": "../../etc/passwd"`	Lista de nomes permitidos; nunca usar o nome recebido como caminho

11.6 SSRF

Quando o payload contém uma URL que o servidor vai buscar (foto de perfil por link, webhook, importação), o atacante aponta para dentro da rede: serviços internos ou o endereço de metadados da nuvem. Foi o mecanismo do caso Capital One.

- Aceitar só `https` e só hosts de uma lista de permissão, quando possível.
- Resolver o nome e recusar endereços privados, de loopback e de metadados.
- Não seguir redirecionamentos.
- Não devolver ao cliente a resposta bruta obtida.
- Restringir na rede o que o serviço consegue alcançar.

11.7 Validação por contrato

A especificação OpenAPI descreve o formato de cada requisição com JSON Schema. O mesmo contrato serve para documentar, gerar código, validar no gateway e testar.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["produtoId", "quantidade"],
  "properties": {
    "produtoId": { "type": "string", "format": "uuid" },
    "quantidade": { "type": "integer", "minimum": 1, "maximum": 100 },
    "observacao": { "type": "string", "maxLength": 200 }
  }
}
```

`additionalProperties: false` é a linha que barra o mass assignment.

11.8 Mensagens de erro

O erro deve ajudar o cliente legítimo sem ensinar o atacante. Dizer qual campo está inválido é aceitável. Stack trace, nome de tabela, versão de biblioteca e consulta SQL nunca saem na resposta: ficam no log, ligados a um identificador que o cliente pode informar ao suporte. A RFC 9457 define um formato padrão para erros de API (Problem Details).

11.9 Em microsserviços

- Cada serviço valida suas entradas, mesmo vindas de outro serviço interno. Um serviço comprometido não pode envenenar os demais.
- Mensagens em filas e eventos são payload como qualquer outro.

- Respostas de APIs de terceiros também são entrada não confiável (risco API10): validar formato, impor timeout e limitar tamanho.

11.10 Vantagens e desvantagens

Vantagens	Desvantagens e riscos
Bloqueia classes inteiras de ataque antes da regra de negócio	Schema e código precisam evoluir juntos
Dados mais limpos no banco, menos erros em produção	Regra rígida demais rejeita entrada legítima (nomes com acento, apóstrofo)
O contrato documenta e testa a API ao mesmo tempo	Sanitização agressiva pode corromper o dado
Erros claros para quem integra	Validação não confere permissão: payload válido ainda pode pedir objeto alheio

12. Arquitetura segura para APIs e microsserviços

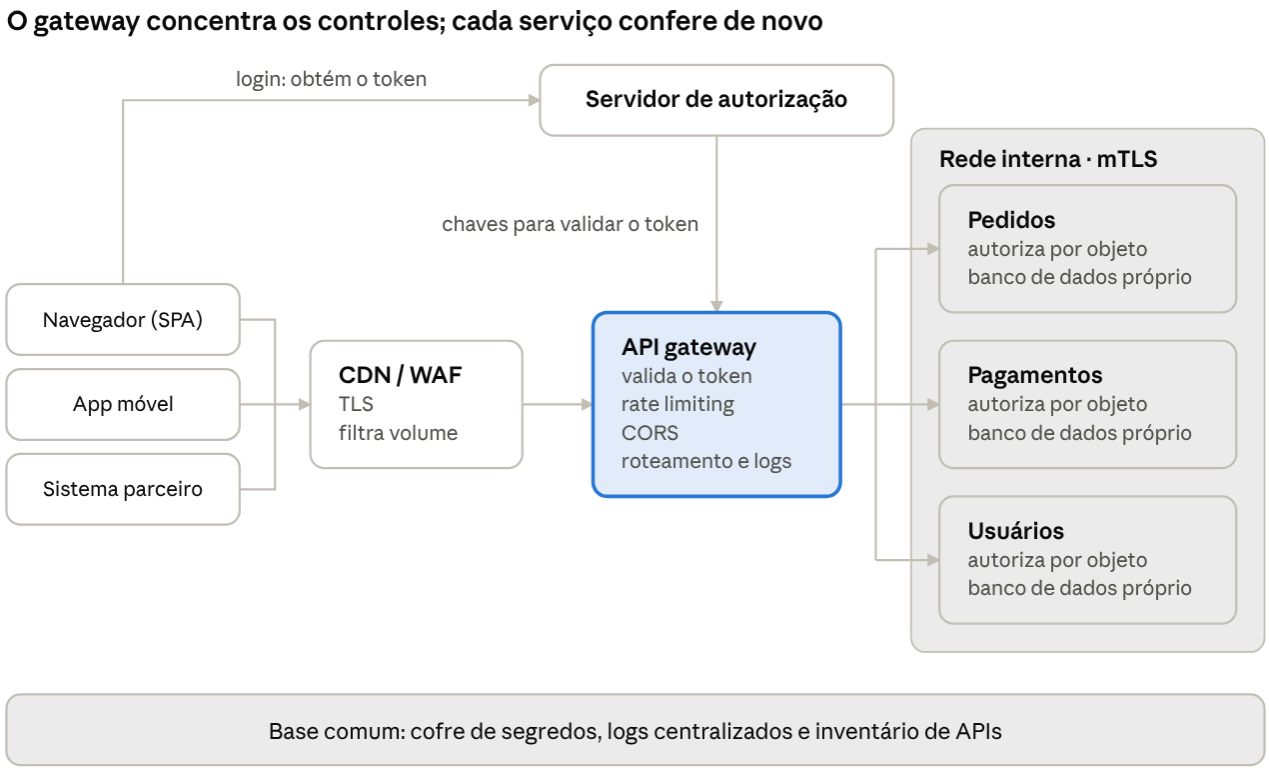
A arquitetura segura concentra os controles comuns na borda (gateway) e repete os controles essenciais dentro de cada serviço, sem confiar na rede interna.

12.1 Princípios

Princípio	Significado	Na prática
Defesa em profundidade	Várias camadas independentes	Gateway valida o token e o serviço valida de novo
Menor privilégio	Cada parte só acessa o mínimo necessário	Token com escopo restrito; serviço com usuário de banco limitado
Zero Trust	Nunca confiar, sempre verificar	Chamada interna também é autenticada e autorizada
Negar por padrão	O que não foi liberado está proibido	Rota nova nasce exigindo autenticação
Falhar fechado	Em caso de erro, negar	Serviço de autorização fora do ar resulta em 503, não em acesso livre

Princípio	Significado	Na prática
Superfície mínima	Expor só o necessário	Serviços internos sem endereço público
Segurança desde o projeto	Decidir os controles antes de codificar	Contrato OpenAPI já traz autenticação e limites

12.2 Visão geral



arquitetura de referência · clientes, borda, gateway e 3 serviços

Só o gateway tem endereço público. Ele barra o que é comum a todas as rotas, e cada serviço decide sozinho se o usuário pode acessar aquele objeto. Dentro da rede interna, os serviços se identificam por mTLS.

12.3 Componentes

Componente	Papel na segurança	Exemplos
CDN / WAF	Absorve volume, filtra ataques conhecidos	Cloudflare, AWS WAF, Akamai

Componente	Papel na segurança	Exemplos
API gateway	Ponto único de entrada: TLS, autenticação, rate limit, CORS, roteamento, logs	Kong, NGINX, AWS API Gateway, Apigee
BFF (Backend for Frontend)	Uma API de borda por tipo de cliente; guarda tokens no servidor em vez do navegador	Serviço próprio da equipe de front-end
Servidor de autorização	Autentica usuários e emite tokens	Keycloak, Auth0, Microsoft Entra ID
Service mesh	mTLS e políticas entre serviços, sem mudar o código	Istio, Linkerd
Cofre de segredos	Guarda e rotaciona senhas, chaves e certificados	HashiCorp Vault, AWS Secrets Manager
Motor de políticas	Regras de autorização fora do código	Open Policy Agent
Observabilidade	Logs, métricas e rastreamento centralizados	OpenTelemetry, Grafana, ELK

12.4 API gateway

Vantagens	Desvantagens e riscos
Política única de autenticação, CORS e rate limit	Ponto único de falha se não houver redundância
Serviços internos ficam fora da internet	Falsa segurança: o serviço passa a confiar em tudo que vem "de dentro"
Um lugar só para logs de acesso e métricas	Pode virar gargalo de desempenho
Facilita inventário e versionamento	Lógica de negócio acaba migrando para o gateway se não houver disciplina

O gateway faz a autorização **grossa** (o token é válido, tem o escopo da rota). A autorização **fina** (este pedido é deste usuário) só o serviço consegue fazer, porque só ele conhece os dados.

12.5 Tráfego norte-sul e leste-oeste

- **Norte-sul:** entre o mundo externo e o sistema. É o que o gateway protege.

- **Leste-oeste:** entre serviços, dentro do sistema. Em microsserviços é a maior parte do tráfego e, historicamente, a menos protegida.

Para o tráfego leste-oeste:

- **mTLS.** No TLS comum, só o servidor apresenta certificado. No TLS mútuo, os dois lados apresentam. Cada serviço prova quem é, e a conexão é cifrada.
- **Service mesh.** Um proxy ao lado de cada serviço (sidecar) cuida do mTLS, da rotação de certificados e de políticas como "só o serviço de pedidos pode chamar o de pagamentos".
- **Segmentação de rede.** Regras que impedem um serviço de alcançar o que não precisa. Limita o movimento lateral de um invasor.

12.6 Zero Trust

O modelo antigo é o do castelo: muro forte e confiança total do lado de dentro. Zero Trust, descrito no NIST SP 800-207, parte do pressuposto de que a rede interna já pode estar comprometida. Toda requisição é autenticada, autorizada e cifrada, venha de onde vier. O NIST SP 800-228 aplica o mesmo raciocínio a APIs e descreve gateways, service meshes e sidecars como pontos de aplicação dos controles.

12.7 Segredos

Senhas de banco, chaves de assinatura de token e chaves de API de terceiros não ficam no código nem no repositório. Ficam em um cofre, são injetadas em tempo de execução e são rotacionadas. Cada serviço tem as suas credenciais: segredo compartilhado entre todos os serviços transforma a invasão de um na invasão de todos. O Tema 3 aprofunda este ponto.

12.8 Logs e monitoramento

Registrar	Nunca registrar
Quem (usuário, cliente, IP), o quê (rota, verbo), quando, resultado (código)	Tokens, senhas, chaves
Falhas de autenticação e de autorização	Corpo completo com dados pessoais
Rejeições por validação e por rate limit	Números de cartão, documentos
Identificador de correlação que acompanha a requisição por todos os serviços	Segredos de configuração

Alertas úteis: pico de respostas 401, 403 ou 429; um mesmo token acessando muitos IDs diferentes em sequência; volume de dados por cliente fora do padrão. No caso T-Mobile, a extração durou semanas até ser percebida.

12.9 TLS e cabeçalhos

- HTTPS em tudo, com TLS 1.2 no mínimo e 1.3 de preferência.
- `Strict-Transport-Security` para o navegador nunca usar HTTP.
- `X-Content-Type-Options: nosniff` para o navegador respeitar o tipo declarado.
- `Cache-Control: no-store` em respostas com dados pessoais.
- `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` em respostas de API.
- Remover cabeçalhos que revelam tecnologia e versão, como `X-Powered-By` e `Server`.

12.10 Inventário e versionamento

- Catálogo de todas as APIs, com dono, versão, dados expostos e ambiente.
- A especificação OpenAPI é a fonte da verdade; o que não está nela não deveria estar no ar.
- Versões antigas têm data para sair e saem.
- Ambientes de teste e homologação não ficam expostos com dados reais.
- **API sombra** é a que está no ar sem ninguém saber. **API zumbi** é a versão antiga que deveria ter sido desligada. O caso Optus foi uma API zumbi.

12.11 Resiliência

Disponibilidade também é segurança. Em microsserviços, uma falha se propaga em cascata se nada a contiver.

Padrão	O que faz
Timeout	Desiste de uma chamada que demora demais
Circuit breaker	Para de chamar um serviço que está falhando e tenta de novo depois
Bulkhead	Isola recursos para que um cliente ou rota não esgote tudo
Retry com espera crescente	Repete com intervalos maiores, sem amplificar a sobrecarga

13. Aplicação prática e código

Os trechos abaixo formam uma API de pedidos em Node.js com Express que aplica os cinco itens do escopo; o código foi executado e os resultados da seção 13.8 são a saída real dos testes.

Versões usadas: Node.js 22, express 5.3, helmet 8.3, cors 2.8, express-rate-limit 8.7, zod 4.6 e jose 6.2. A escolha de Node.js é só para o exemplo: os mesmos controles existem em Spring Boot, ASP.NET, Django e Laravel.

```
npm install express helmet cors express-rate-limit zod jose
```

13.1 A falha mais comum: BOLA

O token é válido, o usuário está autenticado, e mesmo assim ele lê o pedido de outra pessoa.

```
// VULNERÁVEL: busca só pelo id e devolve o registro inteiro
app.get('/inseguro/pedidos/:id', autenticar, (req, res) => {
  const pedido = pedidos.find((p) => p.id === req.params.id);
  if (!pedido) return res.status(404).json({ erro: 'nao_encontrado' });
  res.json(pedido);
});
```

```
// CORRIGIDO: busca pelo id E pelo dono; devolve só os campos permitidos
const paraResposta = ({ id, produto, quantidade }) => ({ id, produto,
quantidade });

app.get('/pedidos/:id', autenticar, (req, res) => {
  const pedido = pedidos.find(
    (p) => p.id === req.params.id && p.usuarioId === req.usuario.id
  );
  if (!pedido) return res.status(404).json({ erro: 'nao_encontrado' });
  res.set('Cache-Control', 'no-store').json(paraResposta(pedido));
});
```

Duas correções em uma rota: o dono entra na busca (resolve API1) e a resposta passa por um filtro de campos (resolve a exposição excessiva, API3). A resposta para pedido alheio é 404, não 403, para não confirmar que o pedido existe. Com banco SQL, a mesma ideia é `WHERE id = ? AND usuario_id = ?`, sempre com consulta parametrizada.

13.2 Autenticação com JWT

```
import { SignJWT, jwtVerify } from 'jose';

const SEGREDO = new TextEncoder().encode(process.env.JWT_SECRET); // vem do
ambiente
const EMISSOR = 'https://auth.exemplo.com';
const AUDIENCIA = 'api-pedidos';

// Emissão: token curto, com emissor, audiência e expiração
const token = await new SignJWT({ papel: u.papel })
  .setProtectedHeader({ alg: 'HS256' })
  .setSubject(u.id).setIssuer(EMISSOR).setAudience(AUDIENCIA)
  .setIssuedAt().setExpirationTime('15m')
  .sign(SEGREDO);

// Validação: middleware aplicado às rotas protegidas
async function autenticar(req, res, next) {
  const [tipo, token] = (req.headers.authorization ?? '').split(' ');
  if (tipo !== 'Bearer' || !token) {
    return res.status(401).json({ erro: 'nao_autenticado' });
  }
  try {
    const { payload } = await jwtVerify(token, SEGREDO, {
      algorithms: ['HS256'],   // lista fixa: o token não escolhe o algoritmo
      issuer: EMISSOR,
      audience: AUDIENCIA,     // a expiração é verificada automaticamente
    });
    req.usuario = { id: payload.sub, papel: payload.papel };
    next();
  } catch {
    res.status(401).json({ erro: 'token_invalido' });
  }
}
```

O que observar: o segredo vem de variável de ambiente; o token dura 15 minutos; a lista `algorithms` impede o ataque do `alg: none`; a identidade do usuário sai do token, nunca do corpo da requisição.

Em microsserviços, troca-se HS256 por RS256 ou ES256: o servidor de autorização assina com a chave privada e cada serviço valida com a chave pública, obtida do endereço JWKS do emissor.

13.3 Autorização por função

```
const exigirPapel = (papel) => (req, res, next) =>
  req.usuario.papel === papel ? next() : res.status(403).json({ erro:
'proibido' });

app.delete('/admin/pedidos/:id', autenticar, exigirPapel('admin'), (req, res)
=> {
  res.status(204).end();
});
```

13.4 Rate limiting

```
import { rateLimit } from 'express-rate-limit';

const limiteGeral = rateLimit({
  windowMs: 60_000, limit: 100,            // 100 requisições por minuto
  standardHeaders: 'draft-8', legacyHeaders: false,
  message: { erro: 'limite_excedido' },
});
const limiteLogin = rateLimit({
  windowMs: 15 * 60_000, limit: 5,         // 5 tentativas a cada 15 minutos
  standardHeaders: 'draft-8', legacyHeaders: false,
  message: { erro: 'muitas_tentativas' },
});

app.use(limiteGeral);
app.post('/login', limiteLogin, validar(LoginSchema), /* ... */);
```

Por padrão a biblioteca conta por IP e guarda o contador na memória do processo. Em produção com várias instâncias, usar um armazenamento compartilhado (Redis) e, nas rotas autenticadas, contar por usuário com a opção `keyGenerator`. Atrás de um proxy ou gateway, configurar `trust proxy` no Express com o número exato de proxies.

13.5 CORS

```
// VULNERÁVEL: reflete qualquer origem e ainda permite credenciais
app.use(cors({ origin: true, credentials: true }));
```

```
// CORRIGIDO: lista fixa de origens, verbos e cabeçalhos
const ORIGENS_PERMITIDAS = ['https://loja.exemplo.com'];

app.use(cors({
  origin: ORIGENS_PERMITIDAS,
  methods: ['GET', 'POST', 'PATCH', 'DELETE'],
  allowedHeaders: ['Authorization', 'Content-Type'],
  maxAge: 600,
}));
```

A API autentica por token no cabeçalho, então `credentials` fica desligado. Para uma origem fora da lista, a resposta sai sem `Access-Control-Allow-Origin` e o navegador bloqueia a leitura.

13.6 Validação de payload

```
import { z } from 'zod';

const NovoPedidoSchema = z.strictObject({          // campo desconhecido é
rejeitado
  produto: z.string().trim().min(1).max(80),
  quantidade: z.number().int().min(1).max(100),
});

const validar = (schema) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) {
    return res.status(400).json({
      erro: 'payload_invalido',
      campos: r.error.issues.map((i) => i.path.join('.') || '(raiz)'),
    });
  }
  req.dados = r.data;                               // só dados validados
seguem adiante
  next();
};

app.post('/pedidos', autenticar, validar(NovoPedidoSchema), (req, res) => {
  const pedido = {
    id: String(1001 + pedidos.length),
    usuarioId: req.usuario.id,                      // dono vem do token, não
do corpo
    ...req.dados,
    custoInterno: 0,
```

```
  };
  pedidos.push(pedido);
  res.status(201).json(paraResposta(pedido));
});
```

`z.strictObject` é o que barra o mass assignment. `.trim()` é a sanitização. Os tipos barram a injeção NoSQL: um objeto no lugar de um texto é recusado.

13.7 Cabeçalhos, limite de corpo e erros

```
import helmet from 'helmet';

app.disable('x-powered-by');                 // não revela a tecnologia
app.use(helmet());                           // HSTS, nosniff e outros
cabeçalhos
app.use(express.json({ limit: '10kb' }));    // corpo maior que isso resulta
em 413

// Último middleware: detalhe no log, mensagem genérica na resposta
app.use((err, req, res, next) => {
  if (err.type === 'entity.too.large') return res.status(413).json({ erro:
'corpo_grande_demais' });
  if (err.type === 'entity.parse.failed') return res.status(400).json({ erro:
'json_invalido' });
  console.error(err);
  res.status(500).json({ erro: 'erro_interno' });
});
```

A ordem dos middlewares é parte da segurança: cabeçalhos, limite de corpo, CORS, rate limit, autenticação, autorização, validação e só então a regra de negócio.

13.8 Resultado dos testes

Cada linha é uma requisição feita com curl contra a API em execução. Ana e Beto são dois clientes; o pedido 1001 é de Ana e o 1002 é de Beto.

Teste	Requisição	Resposta	Controle que atuou
Sem token	`GET /pedidos/1001`	401 `nao_autenticado`	Autenticação
Ana lê o próprio pedido	`GET /pedidos/1001`	200, só `id`, `produto`, `quantidade`	Filtro de saída

Teste	Requisição	Resposta	Controle que atuou
Rota vulnerável: Ana lê o pedido de Beto	`GET /inseguro/pedidos/1002`	200, com `usuarioId` e `custoInterno`	Nenhum: é a falha BOLA
Rota corrigida: Ana tenta o pedido de Beto	`GET /pedidos/1002`	404 `nao_encontrado`	Autorização por objeto
Cliente chama rota de administrador	`DELETE /admin/pedidos/1001`	403 `proibido`	Autorização por função
Mass assignment	`POST /pedidos` com `custoInterno` e `usuarioId` extras	400 `payload_invalido`	Schema estrito
Tipo errado	`"quantidade": "1"`	400, campo `quantidade`	Validação de tipo
Injeção NoSQL no login	`"senha": {"$ne": null}`	400, campo `senha`	Validação de tipo
Token adulterado	Um caractere a mais na assinatura	401 `token_invalido`	Verificação de assinatura
Token com `alg: none`	Token forjado sem assinatura, papel `admin`	401 `token_invalido`	Lista fixa de algoritmos
Corpo de 20 KB	`POST /pedidos`	413 `corpo_grande_demais`	Limite de tamanho
Preflight de origem permitida	`OPTIONS` com `Origin: https://loja.exemplo.com`	204 com `Access-Control-Allow-Origin`	CORS
Preflight de origem desconhecida	`OPTIONS` com `Origin: https://site-malicioso.net`	204 **sem** `Access-Control-Allow-Origin`	CORS
Força bruta no login	Tentativas seguidas com senha errada	401 até a 5ª tentativa da janela, depois 429 com `Retry-After: 900`	Rate limiting

Cabeçalhos reais da resposta bloqueada no login:

```
HTTP/1.1 429 Too Many Requests
RateLimit-Policy: "5-in-15min"; q=5; w=900
RateLimit: "5-in-15min"; r=0; t=900
Retry-After: 900
```

Para uma demonstração ao vivo de dois minutos, as linhas 3 e 4 da tabela bastam: a mesma requisição, com o mesmo token, devolve os dados de Beto na rota vulnerável e 404 na corrigida.

13.9 O que o exemplo simplifica

- Os dados ficam em memória e as senhas em texto puro, só para caber no slide. Em sistema real, banco de dados e senha com hash Argon2id ou bcrypt.
- Não há refresh token, revogação nem segundo fator.
- O limite é por IP e em memória; em produção, Redis e chave por usuário.
- O `helmet` aplica uma política de conteúdo pensada para sites; para API pura vale restringir mais, como na seção 12.9.
- Não há gateway nem mTLS: em microsserviços, parte destes controles sobe para o gateway e o restante permanece em cada serviço.

14. Benefícios x malefícios, vantagens x desvantagens

Microsserviços trocam um problema de segurança concentrado por vários problemas menores e distribuídos: o isolamento melhora, mas a superfície de ataque e o custo de operação crescem.

14.1 A arquitetura: monólito x microsserviços, pelo olhar da segurança

Critério	Monólito	Microsserviços
Superfície de ataque	Pequena e conhecida	Grande: cada serviço é uma API
Impacto de uma invasão	Total: um processo, um banco	Contido, se houver isolamento e menor privilégio
Correção de vulnerabilidade	Reimplanta tudo; mais lenta	Reimplanta só o serviço; mais rápida
Consistência dos controles	Um lugar para autenticar e validar	Risco de cada equipe fazer de um jeito
Dados em trânsito	Chamadas em memória	Tudo passa pela rede e precisa de TLS

Critério	Monólito	Microsserviços
Segredos	Poucos	Muitos; exige cofre e rotação
Auditoria	Um log	Logs distribuídos; exige correlação
Dependências	Um conjunto para atualizar	Um conjunto por serviço
Equipe necessária	Menor	Exige maturidade em DevSecOps

14.2 A arquitetura: benefícios e malefícios

Benefícios de APIs REST e microsserviços	Malefícios e riscos
Isolamento de falhas e de invasões	Mais pontos de entrada para atacar
Menor privilégio por serviço e por banco	Rede interna vira vetor de ataque
Correções e atualizações independentes	Configuração errada se multiplica
Escala por serviço, o que ajuda contra sobrecarga	APIs sombra e zumbis por falta de inventário
Contratos explícitos (OpenAPI) que podem ser validados	Complexidade de identidade: usuário e serviço
Padrões abertos e maduros (HTTP, TLS, OAuth, JWT)	Difícil enxergar um ataque que cruza vários serviços
Integração com parceiros, aplicativos e IA	Dados e lógica de negócio diretamente expostos

14.3 Os controles do escopo

Controle	Benefício principal	Custo ou risco principal	Não resolve
Tokens (JWT)	Autenticação sem estado, validação local e rápida	Revogação difícil; conteúdo legível	Autorização por objeto
Rate limiting	Disponibilidade, anti força bruta, controle de custo	Bloqueio de usuário legítimo se mal calibrado	Ataque lento e distribuído; falha de autorização

Controle	Benefício principal	Custo ou risco principal	Não resolve
CORS	Front-end e API em origens separadas sem expor o usuário	Configuração confusa induz a liberar tudo	Clientes fora do navegador; CSRF com cookies
Validação de payload	Barra injeção, mass assignment e dados inválidos	Manutenção de schemas; falso positivo	Pedido válido para recurso alheio
Gateway	Política única e visibilidade	Ponto único de falha; falsa sensação de segurança	Autorização fina dentro do serviço
mTLS e service mesh	Identidade forte entre serviços, tráfego cifrado	Complexidade operacional, gestão de certificados	Quem é o usuário final
Zero Trust	Limita o movimento lateral do invasor	Mais latência e mais configuração	Falha de lógica de negócio

A coluna "Não resolve" é o argumento central da apresentação: **cada controle cobre uma parte, e a segurança vem da soma.**

14.4 Custo de não fazer x custo de fazer

Sem os controles	Com os controles
Vazamento de dados pessoais e sanções da LGPD	Tempo de desenvolvimento maior no início
Indisponibilidade e perda de receita	Alguma latência a mais por requisição
Fraude e abuso de fluxos de negócio	Infraestrutura adicional (gateway, cofre, Redis)
Custo de nuvem inflado por abuso	Equipe precisa ser treinada
Dano à reputação, processos e multas	Ajuste contínuo de limites e regras

14.5 Mitos e fatos

Mito	Fato
"A API é interna, não precisa de autenticação"	Rede interna não é confiável; a API da Optus também não era para uso público

Mito	Fato
"Tem HTTPS, então está segura"	TLS protege o transporte, não decide quem pode ver o quê
"O aplicativo não mostra esse campo"	O atacante lê a resposta da API, não a tela
"CORS bloqueia quem não é do meu site"	CORS só vale no navegador
"JWT é criptografado"	É assinado; qualquer um lê o conteúdo
"O gateway já valida, o serviço não precisa"	Sem defesa em profundidade, uma falha no gateway expõe tudo
"Usamos UUID, ninguém adivinha"	IDs vazam em logs, links e outras respostas
"Rate limit resolve ataque"	Só desacelera; sem autorização, o dado sai do mesmo jeito

15. Conclusões e recomendações para desenvolvedores

A conclusão do tema é que as falhas de API mais graves são simples, e a defesa é um conjunto de hábitos aplicados em toda rota, não um produto.

15.1 Principais conclusões

1. **A API é o perímetro.** Em microsserviços não existe mais "dentro" seguro; cada serviço é uma borda.
2. **Autenticar não é autorizar.** No caso Dell o invasor tinha conta válida de parceiro; o que faltou foi conferir o dono do objeto.
3. **Nenhum controle basta sozinho.** Token, rate limit, CORS e validação cobrem riscos diferentes e se complementam.
4. **O cliente nunca é confiável.** Toda validação que importa acontece no servidor.
5. **Segurança começa no projeto.** Contrato, modelo de ameaças e política de acesso vêm antes do código.
6. **O que não está inventariado não está protegido.** APIs esquecidas são a porta dos grandes vazamentos.
7. **Visibilidade decide o tamanho do dano.** Sem log e alerta, a extração dura semanas.

15.2 Checklist para cada endpoint

- [ ] Exige autenticação, ou é público por decisão registrada.
- [ ] Valida assinatura, expiração, emissor e audiência do token.

- [ ] Confere se o objeto pedido pertence ao usuário do token.
- [ ] Confere papel ou escopo para operações restritas.
- [ ] Valida corpo, parâmetros de rota e de consulta por schema.
- [ ] Rejeita campos desconhecidos.
- [ ] Devolve só os campos necessários, com formato de resposta explícito.
- [ ] Tem limite de taxa, de tamanho e de paginação.
- [ ] Responde erros sem detalhe interno.
- [ ] Registra acesso e falhas com identificador de correlação.
- [ ] Está descrito na especificação OpenAPI.
- [ ] Tem teste automatizado com um segundo usuário tentando acessar.

15.3 Checklist da arquitetura

- [ ] HTTPS em todo tráfego externo; TLS ou mTLS no interno.
- [ ] Ponto único de entrada, com serviços internos fora da internet.
- [ ] CORS com lista fixa de origens, configurado em um lugar só.
- [ ] Segredos em cofre, fora do repositório, com rotação.
- [ ] Credencial e banco próprios por serviço.
- [ ] Logs centralizados e alertas para 401, 403 e 429.
- [ ] Testes de segurança no pipeline de integração contínua.
- [ ] Inventário de APIs e versões, com data para desligar as antigas.

15.4 Dez recomendações em uma linha

#	Faça	Evite
1	Busque o objeto por ID e dono	Buscar só pelo ID recebido
2	Use tokens curtos, com rotação	Tokens sem expiração
3	Fixe os algoritmos aceitos	Confiar no `alg` do token
4	Valide por lista de permissão	Tentar bloquear padrões ruins
5	Defina o formato de saída de cada rota	Serializar a entidade do banco
6	Limite por usuário e por rota sensível	Um limite único por IP
7	Liste as origens do CORS	Refletir o cabeçalho `Origin`
8	Guarde segredos em cofre	Segredo no código ou no repositório

#	Faça	Evite
9	Valide de novo dentro do serviço	Confiar que o gateway já validou
10	Mantenha o inventário e desligue o que não usa	Deixar `/v1` no ar "por garantia"

15.5 Tendências

- **Agentes de IA como clientes de API.** Aumentam o volume de chamadas automatizadas e pedem escopos mínimos e limites por agente.
- **Código gerado por IA.** O relatório da Akamai de 2026 aponta falhas e configurações erradas chegando à produção sem teste. Revisão e testes de segurança continuam obrigatórios.
- **Tokens vinculados ao cliente.** mTLS e DPoP (RFC 9449) fazem o token roubado não servir em outra máquina.
- **OAuth 2.1.** Consolida as boas práticas e remove fluxos inseguros; ainda é rascunho na IETF.
- **Proteção por comportamento.** Detecção de uso anômalo em tempo de execução, já que os ataques parecem tráfego legítimo.

16. Sugestão de roteiro da apresentação

Para caber em 20 minutos com quatro pessoas, a sugestão é um bloco de cerca de 5 minutos por integrante, seguindo as quatro partes exigidas, com 13 slides.

A divisão de quem fala o quê fica a critério do grupo; a tabela usa letras.

Bloco	Tempo	Quem	Slides	Conteúdo	Seções de apoio
1. Introdução e conceituação	4 min	A	1 a 4	Capa; o que são API REST e microsserviços; por que são alvo (caso Optus ou Dell); onde entra no ciclo de vida	3, 4, 5, 7
2. Funcionamento e arquitetura	6 min	B	5 a 8	OWASP API Top 10 em um slide; diagrama das camadas de defesa; fluxo do token; preflight do CORS e algoritmos de rate limit	6, 8, 9, 10, 12
3. Aplicação prática e código	6 min	C	9 a 11	BOLA vulnerável x corrigido; validação de payload e mass assignment; demonstração ou tabela de resultados	11, 13

Bloco	Tempo	Quem	Slides	Conteúdo	Seções de apoio
4. Conclusão	4 min	D	12 e 13	Vantagens x desvantagens; checklist do desenvolvedor; referências	14, 15

16.1 Slide a slide

#	Título sugerido	O que mostrar
1	Segurança em APIs REST e Microsserviços	Tema, integrantes, disciplina
2	O que é uma API REST e o que são microsserviços	Comparação monólito x microsserviços em uma figura
3	Por que a API virou o alvo	Um caso real e os números de 2026
4	Onde isso entra no ciclo de vida	Linha do requisito à desativação, com um controle por fase
5	OWASP API Security Top 10 (2023)	A lista, com os itens do nosso escopo destacados
6	Defesa em camadas	Diagrama da seção 6.4 ou da 12.2
7	Autenticação por token	As três partes do JWT e o fluxo da requisição
8	Rate limiting e CORS	Token bucket em uma figura; sequência do preflight
9	A falha número 1: BOLA	Código vulnerável e corrigido lado a lado
10	Validação de payload	Requisição de mass assignment e o schema que a barra
11	Testando a API	Tabela de resultados ou demonstração ao vivo
12	Vantagens x desvantagens	Tabela da seção 14.3 resumida
13	Recomendações e referências	Checklist curto e fontes

16.2 Dicas

- Código em slide: no máximo 10 a 12 linhas, fonte grande, com a linha da correção destacada.

- Se fizerem demonstração ao vivo, gravar um vídeo curto de reserva.
- Ensaiar com cronômetro: o limite é de 18 a 22 minutos, e o bloco 2 é o que mais tende a estourar.
- Não detalhar fluxos de OAuth, criptografia nem injeções: são os Temas 2, 3 e 8.

16.3 Perguntas prováveis

Pergunta	Resposta curta
Qual a diferença entre autenticação e autorização?	Autenticação identifica quem é; autorização decide o que pode. 401 x 403.
JWT é seguro? Dá para ler o conteúdo?	É assinado, não cifrado. O conteúdo é legível; a assinatura impede alteração.
Como revogar um JWT?	Vida curta, rotação de refresh token e lista de identificadores revogados.
CORS protege a API de um atacante com curl?	Não. CORS é aplicado pelo navegador para proteger o usuário.
Rate limit por IP é suficiente?	Não. Vários usuários dividem um IP e um atacante usa vários. Combinar com limite por usuário.
O gateway já valida o token; por que validar no serviço?	Defesa em profundidade e Zero Trust: o serviço não deve confiar na rede interna.
Microsserviços são mais seguros que monólito?	Depende. Isolam melhor, mas ampliam a superfície e exigem mais maturidade.
Qual a diferença entre validar e sanitizar?	Validar rejeita o que foge da regra; sanitizar transforma o dado.
O que é BOLA?	Acessar objeto de outro usuário trocando o ID, por falta de checagem de dono.
Por que 404 e não 403 ao negar um objeto alheio?	Para não confirmar que o objeto existe.

17. Glossário

Termo	Significado
ABAC	Controle de acesso baseado em atributos do usuário, do recurso e do contexto
Access token	Token de vida curta enviado em cada requisição para acessar a API
API gateway	Ponto único de entrada que aplica autenticação, limites e roteamento
API sombra	API em produção que não consta no inventário
API zumbi	Versão antiga que deveria ter sido desligada e continua no ar
Bearer token	Token que dá acesso a quem o portar, sem outra prova
BFF	Backend for Frontend: API de borda feita para um tipo de cliente
BFLA	Broken Function Level Authorization: usuário chama função que não é do seu perfil
BOLA	Broken Object Level Authorization: usuário acessa objeto de outro trocando o ID
Claim	Cada informação contida em um JWT
CORS	Cross-Origin Resource Sharing: exceção controlada à política de mesma origem
CSRF	Ataque em que um site faz o navegador da vítima enviar requisição autenticada a outro
DAST	Teste de segurança com a aplicação em execução
Escopo	Permissão específica concedida a um token
Idempotente	Operação que produz o mesmo resultado se repetida
IDOR	Nome anterior e mais geral para o que a OWASP chama de BOLA
JWKS	Endereço onde o emissor publica as chaves públicas de assinatura

Termo	Significado
JWT	JSON Web Token: token assinado que carrega dados do usuário
Mass assignment	Gravar campos enviados pelo cliente sem filtrar quais são permitidos
mTLS	TLS mútuo: cliente e servidor apresentam certificado
OAuth 2.0	Padrão de autorização delegada por tokens
OpenAPI	Formato de descrição de APIs REST legível por máquina
Payload	Corpo de dados de uma requisição ou resposta
PKCE	Extensão do OAuth que protege o código de autorização contra interceptação
Preflight	Requisição `OPTIONS` que o navegador envia antes de uma chamada entre origens
Rate limiting	Limite de requisições por cliente em um intervalo
RBAC	Controle de acesso baseado em papéis
Refresh token	Token de vida longa usado para obter novos access tokens
SAST	Análise estática de segurança no código-fonte
Service mesh	Camada de infraestrutura que gerencia a comunicação entre serviços
Sidecar	Proxy que roda ao lado de cada serviço em um service mesh
SSRF	Ataque que faz o servidor requisitar um endereço escolhido pelo atacante
Stateless	Sem estado: o servidor não guarda sessão entre requisições
STRIDE	Método de modelagem de ameaças em seis categorias
Throttling	Atrasar ou enfileirar requisições acima do limite
WAF / WAAP	Firewall de aplicação web; proteção de aplicações web e APIs

Termo	Significado
Zero Trust	Modelo em que nenhuma requisição é confiável por sua origem na rede

18. Referências

As referências estão no formato da ABNT NBR 6023 e divididas em dois grupos: as páginas consultadas na pesquisa deste documento e as obras de base do tema, cujos links o grupo deve abrir antes de citar nos slides.

18.1 Fontes consultadas nesta pesquisa (acesso em 9 out. 2026)

AKAMAI. **AI Transformation at Risk: APIs Emerge as the Primary Attack Surface, Akamai Research Finds**. Cambridge, 17 mar. 2026. Disponível em: https://www.akamai.com/newsroom/press-release/ai-transformation-at-risk-apis-emerge-as-the-primary-attack-surface-akamai-research-finds. Acesso em: 9 out. 2026.

ABRAMS, Lawrence. **Dell API abused to steal 49 million customer records in data breach**. BleepingComputer, maio 2024. Disponível em: https://www.bleepingcomputer.com/news/security/dell-api-abused-to-steal-49-million-customer-records-in-data-breach/. Acesso em: 9 out. 2026.

BANK INFO SECURITY. **Australia Optus 2022 Data Breach 'Not Highly Sophisticated'**. Jun. 2024. Disponível em: https://www.bankinfosecurity.com/australia-optus-2022-data-breach-not-highly-sophisticated-a-25594. Acesso em: 9 out. 2026.

CIELO. **API Pix: requisitos de segurança**. Disponível em: https://docs.cielo.com.br/pix-can-cbk/docs/requisitos-de-seguranca. Acesso em: 9 out. 2026.

IETF. **RateLimit header fields for HTTP**. Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, maio 2026. Disponível em: https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/. Acesso em: 9 out. 2026.

IETF. **The OAuth 2.1 Authorization Framework**. Internet-Draft draft-ietf-oauth-v2-1. Disponível em: https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/. Acesso em: 9 out. 2026.

LODDERSTEDT, T. et al. **RFC 9700: Best Current Practice for OAuth 2.0 Security**. IETF, jan. 2025. Disponível em: https://www.rfc-editor.org/info/rfc9700. Acesso em: 9 out. 2026.

NIST. **NIST Releases SP 800-228: Guidelines for API Protection for Cloud-Native Systems**. Gaithersburg, 27 jun. 2025. Disponível em: https://csrc.nist.gov/News/2025/nist-releases-sp-800-228. Acesso em: 9 out. 2026.

OWASP. **OWASP API Security Project**. Disponível em: https://owasp.org/www-project-api-security/. Acesso em: 9 out. 2026.

OWASP. **OWASP Top 10 API Security Risks – 2023**. Disponível em: https://owasp.org/API-Security/editions/2023/en/0x11-t10/. Acesso em: 9 out. 2026.

OWASP. **OWASP Top 10:2025**. Disponível em: https://owasp.org/Top10/2025/. Acesso em: 9 out. 2026.

T-MOBILE US. **Form 10-Q: quarter ended September 30, 2023**. U.S. Securities and Exchange Commission, 2023. Disponível em: https://www.sec.gov/Archives/edgar/data/1283699/000128369923000151/Financial_Report.xlsx. Acesso em: 9 out. 2026.

TECHREPUBLIC. **Optus and Medibank Data Breach Cases Allege Cyber Security Failures**. 2024. Disponível em: https://www.techrepublic.com/article/optus-medibank-data-breaches-australia/. Acesso em: 9 out. 2026.

18.2 Obras de base do tema

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.

CHANDRAMOULI, R. **Security Strategies for Microservices-based Application Systems**. Gaithersburg: NIST, 2019. (NIST SP 800-204). Disponível em: https://csrc.nist.gov/pubs/sp/800/204/final.

CHANDRAMOULI, R.; BUTCHER, Z. **Guidelines for API Protection for Cloud-Native Systems**. Gaithersburg: NIST, 2025. (NIST SP 800-228). Disponível em: https://csrc.nist.gov/pubs/sp/800/228/final.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado em Ciência da Informação e Computação) – University of California, Irvine, 2000. Disponível em: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

HARDT, D. (ed.). **RFC 6749: The OAuth 2.0 Authorization Framework**. IETF, out. 2012. Disponível em: https://www.rfc-editor.org/rfc/rfc6749.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519: JSON Web Token (JWT)**. IETF, maio 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519.

LEWIS, James; FOWLER, Martin. **Microservices: a definition of this new architectural term**. 25 mar. 2014. Disponível em: https://martinfowler.com/articles/microservices.html.

NOTTINGHAM, M.; FIELDING, R. **RFC 6585: Additional HTTP Status Codes**. IETF, abr. 2012. Disponível em: https://www.rfc-editor.org/rfc/rfc6585.

NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, jul. 2023. Disponível em: https://www.rfc-editor.org/rfc/rfc9457.

OWASP. **REST Security Cheat Sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html.

ROSE, S. et al. **Zero Trust Architecture**. Gaithersburg: NIST, 2020. (NIST SP 800-207). Disponível em: https://csrc.nist.gov/pubs/sp/800/207/final.

SHEFFER, Y.; HARDT, D.; JONES, M. **RFC 8725: JSON Web Token Best Current Practices**. IETF, fev. 2020. Disponível em: https://www.rfc-editor.org/rfc/rfc8725.

WHATWG. **Fetch Standard**: CORS protocol. Disponível em: https://fetch.spec.whatwg.org/.

18.3 Limites desta pesquisa

- Os números da seção 5.3 vêm do comunicado de imprensa da Akamai; o relatório completo exige cadastro e não foi lido.
- Os detalhes do caso Dell são o relato do autor do ataque à imprensa; a Dell não publicou a causa técnica.
- O dado de 9,5 milhões de clientes da Optus é o citado pelo regulador australiano; a imprensa da época falava em cerca de 10 milhões.
- O caso Capital One e as datas da linha do tempo anteriores a 2019 são de conhecimento consolidado e não foram reconferidos em fonte primária nesta pesquisa.
- O manual da API Pix foi lido pela documentação de uma instituição participante, não pelo documento original do Banco Central.